

Salesforce in the DP Syntropy Pipeline — Handover (2026-06-29)

Where things stand

Every Salesforce node in the n8n account that I could reach is now pointed at "Salesforce account 2," a dedicated Developer Edition sandbox created specifically so this project never has to touch the org with real company data again. That covers Action Approval Handler (Create a Task, Find Existing Lead, Create Lead, Add Existing Lead To Campaign, Add New Lead To Campaign), Brief Approval Handler (Create Salesforce Campaign), and Intelligent Automation (a dead "Perform a query" node, swapped only as a precaution). Three workflows — SC-AI Orchestration Backbone v2, My workflow 2, and Error Workflow — have MCP access switched off in n8n's settings, so they couldn't be audited. By explicit instruction, those three are out of scope and shouldn't be raised again.

Lead creation and dedup are the one piece of this integration that has actually been proven against the new org. A live test today created a real Lead and, on a repeat run, correctly found that same Lead via SOQL instead of duplicating it. That confirms the credential swap works end-to-end for the write path and the query path alike. Both runs used a clearly fake placeholder Salesforce Campaign ID, so the Add-to-Campaign and Create-a-Task steps failed on a malformed-ID error — expected, and not a sign anything is broken.

The real gap, and why it exists

No Salesforce Campaign has ever been created by this pipeline, in either org, ever. The `campaigns` table in Postgres is empty except for today's two disposable test rows. That matters because Action Approval Handler's Lead-to-Campaign-membership and Task-creation steps both read their target Campaign ID from that table — with no real Campaign ID to point at, those two steps can never succeed on real data, regardless of which Salesforce org they're aimed at.

Tracing Brief Approval Handler's wiring explains why. The chain runs Get Config → Select Campaign Template → Insert Campaign → (Create Salesforce Campaign in parallel with Gather Channel Context) → Mark Salesforce Synced. This whole chain, including the AI channel recommender, is brand-new — it shipped today as part of Phase 5 — and every real (non-test) brief currently sitting in `pipeline_runs` predates it. So the emptiness of `campaigns` isn't evidence of a broken integration; it's evidence that the integration has never been exercised by a real approval yet.

There's a known landmine sitting right at the front of that chain, though. Select Campaign Template builds its SQL parameters from `when_urgency`, `coverage_label`, `when_stage`, and `trigger_type` on the approved brief, and if any of those is null, n8n's parameter-splitting silently drops a slot and Postgres throws "there is no parameter \$4" — which would halt the entire chain before a Campaign, any channel recommendation, or any action ever gets created, not just the Salesforce piece. I checked the actual data: the older seed-era briefs (run ids 327–357) do have null `when_urgency`/`when_stage`, but every brief from id 372 onward has all four fields populated. So whatever upstream process feeds `pipeline_runs` now seems to fill these in reliably, and the next real approval will likely sail through — but "likely" isn't "guaranteed," and this is a one-line landmine sitting in a path that's never been tested live.

What to do next, in order

Fix Select Campaign Template first, before anything else touches this chain. The safest fix is to make the query tolerant of nulls directly — either coalesce each value to a non-empty sentinel before the comma-join (so no segment is ever truly empty) or restructure the query to bind parameters individually instead of via one concatenated `queryReplacement` string. This is cheap to fix and expensive to leave, since it can silently kill an entire brief's downstream processing, not just its Salesforce sync.

Once that's fixed, run a real end-to-end test of the Campaign chain — not just Lead sync. Insert a disposable, clearly-prefixed test brief into `pipeline_runs` (same discipline as the Lead-sync tests), approve it through Brief Approval Handler, and confirm a real Campaign appears in the sandbox org and that `salesforce_campaign_id` actually gets written back into the `campaigns` row by Mark Salesforce Synced. This has never been verified, and it's the only way to know the chain works rather than just looks wired correctly.

With a real synced Campaign ID in hand, go back to Action Approval Handler and rerun the Lead-sync test using that real ID instead of a placeholder. That's the only way to ever prove Add-to-Campaign and Create-a-Task actually succeed — they have a 100% failure rate so far, but only because nothing has ever given them a real Campaign ID to work with.

After that, two lower-priority items are worth a look but aren't blocking anything: Intelligent Automation's "Perform a query" Salesforce node is dead code (disconnected from any input, and even if it were connected, it has an invalid `resource` parameter) — worth deleting or fixing whenever that workflow gets attention. And separately, Lead dedup in this pipeline is done by manual SOQL search-then-create-or-reuse because the org has no custom External ID field and `raw_payload.buying_group` contacts have no real email addresses yet — if email enrichment ever gets added upstream, a proper `upsert` against an External ID field would be more robust than the current name+company match.

Last thing: today's test Lead and the four disposable Postgres rows (actions 102/103, pipeline_runs 390/391, campaigns 10/11) are still sitting in the sandbox for you to look at. Clean them up whenever you're done checking.